

Chat System — Data Models

1. Storage Layer Recap

Earlier, we selected:

None

Key-Value Store

as the primary storage solution for chat history because it provides:

- Low latency
- Horizontal scalability
- Efficient handling of massive chat data

2. Most Important Data in Chat System

The most critical data is:

None

Message Data

because the entire chat experience depends on:

- Correct message delivery
- Proper message ordering
- Fast retrieval

3. Message Table for One-on-One Chat

message	
message_id	bigint
message_from	bigint
message_to	bigint
content	text
created_at	timestamp

The message table stores messages exchanged between two users.

4. Primary Key in One-on-One Chat

The primary key is:

None

message_id

5. Why message_id is Important

message_id determines:

- Message ordering
- Message uniqueness
- Retrieval sequence

6. Why created_at Cannot Be Used

Using:

```
None  
created_at timestamp
```

alone is unreliable.

Reason

Two messages can be created at exactly the same time.

Example:

```
None  
Message A → 10:30:15.001  
  
Message B → 10:30:15.001
```

This creates ambiguity in ordering.

7. Solution

Instead of timestamps:

```
None  
message_id
```

is used for strict sequencing.

8. Message Table for Group Chat

group_message	
channel_id	bigint
message_id	bigint
user_id	bigint
content	text
created_at	timestamp

Group chat requires a different table design.

9. Composite Primary Key

The composite primary key is:

```
None  
(channel_id, message_id)
```

10. Meaning of channel_id

Here:

```
None  
channel = group
```

Both terms represent the same concept.

11. Why `channel_id` is the Partition Key

Group chat operations usually happen within one channel.

Examples:

- Fetch group messages
- Send message to group
- Search within group

Thus:

```
None  
channel_id
```

becomes the partition key.

12. Benefits of Partitioning by `channel_id`

Efficient Querying

Messages of same group stay together.

Faster Retrieval

Queries access only relevant partition.

Better Scalability

Data distributes naturally across nodes.

13. Message ID Design

Generating:

None

`message_id`

is a very important design challenge.

14. Responsibilities of message_id

message_id must ensure:

- Correct ordering
- Uniqueness
- Scalability

15. Requirements for message_id

Requirement 1: IDs Must Be Unique

No two messages should share same ID.

Requirement 2: IDs Must Be Time-Sortable

Newer messages should have:

None

Higher IDs

than older messages.

This helps maintain chronological order.

16. Approach 1 — Auto Increment

The first idea is:

None

AUTO_INCREMENT

used in MySQL.

17. Problem with AUTO_INCREMENT

Most NoSQL databases:

None

Do NOT support auto_increment

because distributed systems make centralized counters difficult.

18. Why AUTO_INCREMENT is Difficult at Scale

In distributed systems:

- Multiple servers generate messages simultaneously
- Synchronizing global counters becomes expensive
- Creates bottlenecks

19. Approach 2 — Global Sequence Generator

Second approach:

None

Global 64-bit Sequence Generator

Example:

None

Twitter Snowflake

20. Snowflake-Based IDs

Snowflake IDs provide:

- Global uniqueness
- Time ordering
- Distributed scalability

21. Advantages of Global IDs

Unique Across Entire System

No collisions.

Naturally Sortable

Higher IDs correspond to newer messages.

Distributed Generation

Multiple servers can generate IDs independently.

22. Drawback of Global IDs

Global ID generation is:

None

Complex to implement

because it requires:

- Coordination
- Clock synchronization
- Careful infrastructure design

23. Approach 3 — Local Sequence Generator

Final approach:

None

Local Sequence Number Generator

24. What Does Local Mean?

Local IDs are unique only within:

- One chat
OR
- One group/channel

25. Why Local IDs Work

In chat systems:

None

Message ordering only matters INSIDE a conversation

26. Example

One-on-One Chat

Conversation:

None

User A ↔ User B

Message IDs:

None

1, 2, 3, 4...

Only need to be unique inside this chat.

Group Chat

Channel:

None

channel_101

Messages:

None

1, 2, 3, 4...

Again, uniqueness required only within channel.

27. Advantages of Local Sequence IDs

Easier Implementation

No global coordination needed.

Better Performance

Avoids centralized bottlenecks.

Scales Better

Each channel manages its own sequence independently.

Simpler Architecture

Less complexity than global ID generation.

28. Comparison of Message ID Approaches

Approach	Advantages	Disadvantages
AUTO_INCREMENT	Simple	Poor for distributed NoSQL
Global Snowflake ID	Globally unique, sortable	Complex implementation
Local Sequence ID	Simple, scalable	Only locally unique

29. Recommended Approach

The preferred approach is usually:

None

`Local Sequence Generator`

because:

- Message ordering is conversation-specific
- Easier to scale
- Lower operational complexity

One-on-One Chat

Primary key:

None

`message_id`

Group Chat

Composite key:

None

`(channel_id, message_id)`

Key Takeaway

Message ordering is one of the most important problems in chat systems.

`message_id` must satisfy:

- Uniqueness
- Time ordering

Timestamps alone are insufficient because multiple messages may share identical timestamps.

Local sequence generators are commonly preferred because:

- Ordering only matters within a conversation
- They are simpler
- They scale better
- They avoid global coordination complexity